

Table des matières

Labo 03 — Charger les fichiers livrés dans la couche RAW	1
Avant de commencer	2
Étape 1 — Créer l'entrepôt, et voir le double compte d'un fichier chargé deux fois.....	2
Étape 2 — Écrire le chargeur qui ne prend que le neuf et marque son lot	3
Étape 3 — Retrouver dans la RAW le mois livré deux fois, les messages en double et le client effacé.....	9
Étape 4 — Déposer le corpus tel quel, avec son catalogue.....	10
Étape 5 — Versionner le chargeur et le dépôt du corpus.....	12
Récapitulatif	12
Dépannage	13
Annexe — Tout le code, d'un seul tenant.....	14
Annexe — Sur Snowflake	15
Annexe — Sur Databricks.....	17

Labo 03 — Charger les fichiers livrés dans la couche RAW

Labo · Module 3/12 — Formation Data Engineering

Ce que vous allez faire. Charger les fichiers livrés par les sources de Boréal dans la couche RAW de l'entrepôt DuckDB, par un chargeur qu'on relance sans rien doubler ; retrouver dans la RAW ce qu'elle garde exprès ; déposer la documentation dans le bronze. Objectif de fin : 22 tables et 513 857 lignes dans le schéma `raw`, un second passage du chargeur qui ne charge rien, et 35 fichiers de documentation déposés, puis aucun au second passage.

Deux terminaux servent dans ce labo, et chaque étape dit lequel. L'Anaconda Prompt, environnement `formations` actif, sert à Python et à Git. La ligne de commande DuckDB sert au SQL de vérification.

La liste de contrôle du cours :

- ☐ Un extrait chargé à la main deux fois : 854 lignes pour 427 commandes
- ☐ `01_de/ingestion/charger_raw.py` lancé : 1 572 fichiers et 513 857 lignes, puis « à jour » partout au second passage
- ☐ La RAW inspectée : 22 tables, 609 commandes au lot `2023-03_bis`, les doublons gardés

- ☐ `deposer_corpus.py` lancé : 35 fichiers déposés, puis 0
 - ☐ Le chargeur et le dépôt du corpus versionnés
-

Avant de commencer

Ce cours ne demande que le socle du parcours et ce que les Cours 01 et 02 ont laissé : les données de Boréal dans `Projet_Boreal/sources/`, votre dépôt `01_de` et le lac de `plateforme/`. Aucun service à allumer : tout se fait sous Windows, avec Python et DuckDB. La base de l'ERP arrive au Cours 04.

Étape 1 — Créer l'entrepôt, et voir le double compte d'un fichier chargé deux fois

Théorie : `Theorie_03.md`, sections 3 et 4.

Tout le labo part de `Projet_Boreal`, avec les sources générées à l'installation et le bronze du Cours 02. L'entrepôt de Boréal tient en un seul fichier, `plateforme/entrepot/boreal.duckdb`. DuckDB le crée à la première connexion, mais pas son dossier. Dans l'Anaconda Prompt, depuis `Projet_Boreal` :

```
mkdir plateforme\entrepot
duckdb
```

À l'invite `D`, attachez l'entrepôt et faites-en la base courante :

```
-- Attendu : entrepot.
ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot;
USE entrepot;
SELECT current_database();
```

`ATTACH` ouvre le fichier, et le crée s'il n'existe pas ; `USE` fait que les noms de schémas et de tables se lisent dedans. `IF NOT EXISTS` permet de répéter la commande sans erreur. Chaque fois que ce labo rouvre DuckDB, il commence par ces deux lignes.

Avant d'écrire le chargeur, chargez un extrait à la main, pour voir ce qui cloche. Les en-têtes de commande de décembre 2025, tout en texte, avec le nom du fichier d'où vient chaque ligne :

```
-- Attendu : 427 lignes.
CREATE SCHEMA IF NOT EXISTS essai;
CREATE OR REPLACE TABLE essai.vbak AS
SELECT * FROM read_csv('sources/erp/extraits/2025-12/VBAK.csv', all_varchar = true, filename = true);
SELECT COUNT(*) AS lignes FROM essai.vbak;
```

Un chargement réel ajoute, mois après mois. Ajoutez donc le même fichier, comme le ferait un traitement relancé par erreur :

```
-- Attendu : 854 lignes, 427 commandes, et Le fichier sources/erp/extraits/2025-12/VBAK.csv.
INSERT INTO essai.vbak
SELECT * FROM read_csv('sources/erp/extraits/2025-12/VBAK.csv', all_varchar = true, filename = true);
SELECT COUNT(*) AS lignes, COUNT(DISTINCT VBELN) AS commandes, ANY_VALUE(filename) AS fichier
FROM essai.vbak;
```

Rien n’a planté, et chaque commande de décembre est là deux fois. Le CREATE OR REPLACE de la première requête était idempotent : il remplace. L’INSERT ne l’est pas. Un chargeur qui ajoute doit donc se souvenir de ce qu’il a déjà chargé : c’est le rôle du journal des chargements, à l’étape suivante. Retirez l’essai et quittez DuckDB :

```
DROP SCHEMA essai CASCADE;
.exit
```

À retenir — un seul processus à la fois sur l’entrepôt. Tant que la ligne de commande DuckDB tient `boreal.duckdb` ouvert, même en lecture seule, aucun autre programme ne peut l’ouvrir. Quittez toujours DuckDB (`.exit`) avant de lancer un script Python de ce labo.

Étape 2 — Écrire le chargeur qui ne prend que le neuf et marque son lot

Théorie : Theorie_03.md, sections 1, 2, 3 et 4.

Le chargeur est un programme Python : il parcourt les fichiers livrés, demande au journal lesquels sont nouveaux, et ne charge que ceux-là. Il vit dans votre dépôt, `01_de`, dans un dossier `ingestion`. Dans l’Anaconda Prompt, depuis `Projet_Boreal` :

```
mkdir 01_de\ingestion
notepad 01_de\ingestion\charger_raw.py
```

Le Bloc-notes répond qu’il ne trouve pas le fichier et propose de le créer : acceptez. Collez-y le code qui suit, en entier, puis enregistrez et fermez. Ce code ne se tape pas dans la ligne de commande DuckDB : c’est un fichier Python, que Python exécutera.

```
# -*- coding: utf-8 -*-
"""charger_raw.py : Les fichiers livrés par Les sources de Boréal, chargés dans La couche RAW.
```

```
Lancement, depuis Projet_Boreal/01_de : python ingestion/charger_raw.py
"""
```

```
import hashlib
import os
import sys
import time
from datetime import datetime, timezone
from pathlib import Path

import duckdb
```

```

# 01_de/ingestion/charger_raw.py : deux crans plus haut, le dossier du projet.
PROJET = Path(__file__).resolve().parents[2]
ENTREPOT = "plateforme/entrepot/boreal.duckdb"

# Une livraison : la table de la RAW, le motif de ses fichiers, leur forme, et d'où vient le lot.
# dossier : le dossier du mois (2023-03, 2023-03_bis) ; jour : la date du nom de fichier ;
# complet : un export livré en entier, qui remplace le précédent.
LIVRAISONS = [
    ("erp_t005u", "sources/erp/extraits/*/T005U.csv", "csv", "dossier"),
    ("erp_kna1", "sources/erp/extraits/*/KNA1.csv", "csv", "dossier"),
    ("erp_mara", "sources/erp/extraits/*/MARA.csv", "csv", "dossier"),
    ("erp_makt", "sources/erp/extraits/*/MAKT.csv", "csv", "dossier"),
    ("erp_vbak", "sources/erp/extraits/*/VBAK.csv", "csv", "dossier"),
    ("erp_vbap", "sources/erp/extraits/*/VBAP.csv", "csv", "dossier"),
    ("erp_cdhdr", "sources/erp/extraits/*/CDHDR.csv", "csv", "dossier"),
    ("erp_cdpos", "sources/erp/extraits/*/CDPOS.csv", "csv", "dossier"),
    ("erp_zserie", "sources/erp/extraits/*/ZSERIE.csv", "csv", "dossier"),
    ("garanties_clients", "sources/garanties/*/clients.csv", "csv", "dossier"),
    ("garanties_enregistrements", "sources/garanties/*/enregistrements.csv", "csv", "dossier"),
    ("garanties_visites", "sources/garanties/*/visites.csv", "csv", "dossier"),
    ("garanties_consentements", "sources/garanties/*/consentements.csv", "csv", "dossier"),
    ("garanties_reclamations", "sources/garanties/*/reclamations.json", "json", "dossier"),
    ("usine_actifs", "sources/usine/referentiel/actifs.csv", "csv", "complet"),
    ("usine_etiquettes", "sources/usine/referentiel/etiquettes.csv", "csv", "complet"),
    ("usine_bons_travail", "sources/usine/gmao/bons_travail.csv", "csv", "complet"),
    ("meteo_stations", "sources/meteo/stations.csv", "csv", "complet"),
    ("meteo_quotidien", "sources/meteo/quotidien.csv", "csv", "complet"),
    ("meteo_horaire", "sources/meteo/horaire_7060400.csv", "csv", "complet"),
    ("telemetrie_sorties", "sources/telemetrie/sorties_*.jsonl", "jsonl", "jour"),
    ("corpus_index", "sources/corpus/index.csv", "csv", "complet"),
]

# La requête qui lit une liste de fichiers, selon leur forme. Chacune rend la colonne filename.
LECTEURS = {
    "csv": "SELECT * FROM read_csv($fichiers, header = true, all_varchar = true, filename = true)",
    "json": ("SELECT json AS objet, filename "
            "FROM read_json_objects($fichiers, format = 'array', filename = true)"),
    "jsonl": ("SELECT sujet, recu_le, qos, message, filename "
            "FROM read_json($fichiers, format = 'newline_delimited', filename = true, "
            "columns = {'sujet': 'VARCHAR', 'recu_le': 'VARCHAR', 'qos': 'VARCHAR', 'message': 'JSON'})"),
}

```

```

def lot_du(fichier: str, regle: str) -> str:
    chemin = Path(fichier)
    if regle == "dossier":
        return chemin.parent.name
    if regle == "jour":
        return chemin.stem.replace("sorties_", "")
    return "complet"

def empreinte(fichier: str) -> str:
    return hashlib.sha256(Path(fichier).read_bytes()).hexdigest()

def espacer(nombre: int) -> str:
    return f"{nombre:,}".replace(",", " ")

def ouvrir_entrepot():
    Path(ENTREPOT).parent.mkdir(parents=True, exist_ok=True)
    try:
        con = duckdb.connect(ENTREPOT)
    except duckdb.IOException as erreur:
        sys.exit(f"L'entrepôt est occupé : fermez la ligne de commande DuckDB, puis relancez.\n{erreur}")
    con.execute("CREATE SCHEMA IF NOT EXISTS raw")
    con.execute("CREATE SCHEMA IF NOT EXISTS pipeline")
    con.execute("""
        CREATE TABLE IF NOT EXISTS pipeline.chargements (
            fichier VARCHAR,          -- le chemin du fichier, depuis Projet_Boreal
            empreinte VARCHAR,        -- SHA-256 du contenu
            table_raw VARCHAR,
            lot VARCHAR,
            lignes BIGINT,            -- lignes chargées ; 0 pour un fichier vide
            charge_le TIMESTAMPTZ)""")
    return con

def charger(con, table: str, motif: str, forme: str, regle: str) -> tuple:
    """Charge les fichiers nouveaux ou modifiés d'une livraison. Rend (fichiers, lignes)."""
    fichiers = sorted(p.as_posix() for p in Path(".").glob(motif))
    connus = dict(con.execute("""
        SELECT fichier, empreinte FROM pipeline.chargements WHERE table_raw = ?
    """))

```

```

        QUALIFY ROW_NUMBER() OVER (PARTITION BY fichier ORDER BY charge_le DESC) = 1""",
        [table]).fetchall())
a_charger = [(f, e, lot_du(f, regle)) for f in fichiers if connus.get(f) != (e := empreinte(f))]
if not a_charger:
    return 0, 0

source = LECTEURS[forme]
noms = [f for f, _, _ in a_charger]
charge_le = datetime.now(timezone.utc)
con.execute("BEGIN")
try:
    # La table naît de la forme du premier fichier : ses colonnes, en texte, puis le lignage.
    con.execute(f"""
        CREATE TABLE IF NOT EXISTS raw.{table} AS
        SELECT * EXCLUDE (filename), filename AS _fichier, ' ' AS _lot, NOW() AS _charge_le
        FROM ({source}) LIMIT 0""", {"fichiers": noms[:1]})
    con.execute("CREATE OR REPLACE TEMP TABLE a_charger (fichier VARCHAR, empreinte VARCHAR, lot VARCHAR)")
    con.executemany("INSERT INTO a_charger VALUES (?, ?, ?)", a_charger)
    # Un fichier déjà chargé sous une autre empreinte : ses anciennes lignes partent d'abord.
    con.execute(f"DELETE FROM raw.{table} WHERE _fichier IN (SELECT fichier FROM a_charger)")
    con.execute(f"""
        INSERT INTO raw.{table}
        SELECT s.* EXCLUDE (filename), s.filename, a.lot, $charge_le
        FROM ({source}) s JOIN a_charger a ON a.fichier = s.filename""",
        {"fichiers": noms, "charge_le": charge_le})
    con.execute(f"""
        INSERT INTO pipeline.chargements
        SELECT a.fichier, a.empreinte, '{table}', a.lot, COUNT(r._fichier), $charge_le
        FROM a_charger a LEFT JOIN raw.{table} r
            ON r._fichier = a.fichier AND r._charge_le = $charge_le
        GROUP BY ALL""", {"charge_le": charge_le})
    lignes = con.execute(f"SELECT COUNT(*) FROM raw.{table} WHERE _charge_le = ?", [charge_le]).fetchone()[0]
    con.execute("COMMIT")
except Exception:
    con.execute("ROLLBACK")      # qui ouvre la transaction la referme, même quand ça rate
    raise
return len(a_charger), lignes

def main():
    os.chdir(PROJET)
    con = ouvrir_entrepot()

```

```

debut = time.perf_counter()
total_fichiers = total_lignes = 0
for table, motif, forme, regle in LIVRAISONS:
    fichiers, lignes = charger(con, table, motif, forme, regle)
    if fichiers:
        print(f"{table:26} {fichiers:4} fichier(s) chargé(s), {espacer(lignes):>8} lignes")
    else:
        print(f"{table:26}    à jour")
    total_fichiers += fichiers
    total_lignes += lignes
con.close()
print(f"Total : {espacer(total_fichiers)} fichier(s), {espacer(total_lignes)} lignes, "
      f"en {time.perf_counter() - debut:.1f} s.")

if __name__ == "__main__":
    sys.stdout.reconfigure(encoding="utf-8", errors="replace")
    main()

```

Ce que fait chaque morceau :

Morceau	Ce qu'il fait
PROJET	le dossier du projet, calculé depuis l’emplacement du script : le chargeur trouve sources/ d’où qu’on le lance
LIVRAISONS	les 22 livraisons : la table de la RAW, le motif des fichiers, leur forme, la règle du lot
LECTEURS	la requête DuckDB qui lit une liste de fichiers : un CSV tout en texte, un tableau JSON objet par objet, un JSON lines dont le message reste en JSON
empreinte	le SHA-256 du contenu : deux fichiers identiques ont la même, un octet changé la change
pipeline.chargements	le journal : chaque fichier chargé, son empreinte, sa table, son lot, ses lignes, l’heure
a_charger	les fichiers que le journal ne connaît pas, ou connaît avec une autre empreinte
BEGIN ... COMMIT	une transaction : la table et le journal changent ensemble, ou pas du tout

Morceau	Ce qu'il fait
except ... ROLLBACK	qui ouvre une transaction la referme. Sans lui, DuckDB la garde avortée, et la requête suivante répond « Current transaction is aborted » au lieu de laisser passer la vraie cause
DELETE, puis INSERT	un fichier livré de nouveau sous le même nom remplace ses propres lignes
p.as_posix()	le chemin s'écrit avec des barres obliques : le lignage est le même sous Windows et sous Linux

Lancez-le depuis 01_de, d'où se lance tout le code du dépôt :

```
cd 01_de
python ingestion\charger_raw.py
```

Le chargeur affiche une ligne par livraison :

```
erp_t005u          49 fichier(s) chargé(s),      980 lignes
erp_kna1            49 fichier(s) chargé(s),       7 481 lignes
erp_mara            49 fichier(s) chargé(s),       2 459 lignes
erp_makt            49 fichier(s) chargé(s),       4 918 lignes
erp_vbak            49 fichier(s) chargé(s),      26 538 lignes
erp_vbap            49 fichier(s) chargé(s),      81 292 lignes
erp_cdhdr           49 fichier(s) chargé(s),        53 lignes
erp_cdpos           49 fichier(s) chargé(s),        53 lignes
erp_zserie          49 fichier(s) chargé(s),      32 597 lignes
garanties_clients   48 fichier(s) chargé(s),      25 379 lignes
garanties_enregistrements 48 fichier(s) chargé(s),      26 732 lignes
garanties_visites   48 fichier(s) chargé(s),      53 865 lignes
garanties_consentements 48 fichier(s) chargé(s),      32 708 lignes
garanties_reclamations 48 fichier(s) chargé(s),       5 623 lignes
usine_actifs        1 fichier(s) chargé(s),        23 lignes
usine_etiquettes    1 fichier(s) chargé(s),        58 lignes
usine_bons_travail  1 fichier(s) chargé(s),       779 lignes
meteo_stations      1 fichier(s) chargé(s),        11 lignes
meteo_quotidien     1 fichier(s) chargé(s),      17 061 lignes
meteo_horaire       1 fichier(s) chargé(s),      37 068 lignes
telemetrie_sorties  884 fichier(s) chargé(s),     158 146 lignes
corpus_index        1 fichier(s) chargé(s),        33 lignes
Total : 1 572 fichier(s), 513 857 lignes, en 12.5 s.
```

Environ 13 secondes sur le poste de l’auteur ; la durée dépend de votre machine, les comptes non. Les 49 fichiers de l’ERP comptent les 48 mois et 2023-03_bis. Relancez maintenant la même commande :

```
python ingestion\charger_raw.py
```

Chaque livraison répond « à jour », et le total tombe à 0 fichier(s), 0 lignes, en moins d’une seconde. Le journal a reconnu chaque fichier à son chemin et à son empreinte : c’est l’idempotence que l’étape 1 n’avait pas. Revenez dans Projet_Boreal :

```
cd ..
```

Pourquoi — un chargeur en Python plutôt qu’un script SQL. Au Cours 02, un script SQL suffisait : chaque bronze se refaisait en entier. Ici, il faut décider fichier par fichier, se souvenir d’un passage à l’autre et tenir une transaction : c’est le travail d’un programme. Le SQL reste dans les requêtes que Python confie à DuckDB, et le Cours 05 fera de ce script un vrai pipeline.

Étape 3 — Retrouver dans la RAW le mois livré deux fois, les messages en double et le client effacé

Théorie : Theorie_03.md, section 4.

Rouvrez DuckDB (duckdb), cette fois en lecture seule : on regarde, on ne touche à rien.

```
-- Attendu : entrepot, puis 22 tables.
ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot (READ_ONLY);
USE entrepot;
SELECT current_database();
SELECT COUNT(*) AS tables_raw FROM information_schema.tables WHERE table_schema = 'raw';
```

Le journal résume ce qui est entré, livraison par livraison :

```
-- Attendu : 22 lignes, 1 572 fichiers et 513 857 lignes en tout. Des fichiers vides dans erp_cdhdr
-- et erp_cdpos (44 chacun), garanties_visites et garanties_reclamations (1 chacun), telemetrie_sorties (41).
SELECT table_raw, COUNT(*) AS fichiers, SUM(lignes) AS lignes,
       COUNT(*) FILTER (WHERE lignes = 0) AS vides
FROM pipeline.chargements
GROUP BY table_raw
ORDER BY table_raw;
```

Un fichier vide est une livraison comme une autre : le mois sans document de modification donne un fichier d’en-tête seul, le jour sans sortie de véhicule un fichier vide. Le journal les note avec 0 ligne, et un contrôle de volume pourra un jour s’en inquiéter à bon escient. La forme d’une table :

```
-- Attendu : 15 colonnes. Les 12 de l'extrait, de MANDT à WAERK, en varchar ; puis _fichier varchar,
-- _lot varchar et _charge_le timestamp with time zone.
DESCRIBE raw.erp_vbak;
```

Aucun type deviné : le chargeur a tout lu en texte. Les colonnes de lignage montrent aussitôt leur utilité. L'extrait livré deux fois :

```
-- Attendu : 2 lignes, 609 en-têtes au lot 2023-03 et 609 au lot 2023-03_bis.
```

```
SELECT _lot, COUNT(*) AS en_tetes
FROM raw.erp_vbak
WHERE _lot LIKE '2023-03%'
GROUP BY _lot
ORDER BY _lot;
```

```
-- Attendu : 26 538 lignes pour 25 483 commandes.
```

```
SELECT COUNT(*) AS lignes, COUNT(DISTINCT VBELN) AS commandes FROM raw.erp_vbak;
```

Le journal a laissé passer 2023-03_bis, parce qu'il porte un autre nom que 2023-03. La RAW garde les deux, marqués par leur lot : décider lequel compte est une règle métier, et le repérer est le travail d'un contrat (Cours 11). Les doublons de la télémétrie et les versions des réclamations sont gardés de la même façon :

```
-- Attendu : 158 146 messages pour 155 843 identifiants.
```

```
SELECT COUNT(*) AS messages, COUNT(DISTINCT message -> 'message_id') AS identifiants
FROM raw.telemetry_sorties;
```

```
-- Attendu : 5 623 objets pour 3 348 réclamations.
```

```
SELECT COUNT(*) AS objets, COUNT(DISTINCT objet -> 'reclamation_id') AS reclamations
FROM raw.garanties_reclamations;
```

L'opérateur -> lit un champ d'une colonne JSON et le rend en texte. Enfin, ce que garde une couche qui garde tout :

```
-- Attendu : 2 lignes. Au lot 2024-01, un prénom et le statut ACTIF ; au lot 2025-09, un prénom NULL
```

```
-- et le statut EFFACE.
```

```
SELECT _lot, prenom, statut
FROM raw.garanties_clients
WHERE client_id = 'CL0011163'
ORDER BY _lot;
```

Le client a demandé l'effacement de ses données le 2025-09-16. L'extrait de septembre arrive sans ses données personnelles, mais celui de janvier 2024 les contient encore, et la RAW le garde (Théorie 03, section 4). Quittez DuckDB (.exit).

Étape 4 — Déposer le corpus tel quel, avec son catalogue

Théorie : Theorie_03.md, section 5.

Dernière livraison : la documentation, déposée telle quelle. Dans l'Anaconda Prompt, depuis Projet_Boreal :

```
notepad 01_de\ingestion\deposer_corpus.py
```

```
# -*- coding: utf-8 -*-
"""deposer_corpus.py : La documentation de Boréal déposée telle quelle dans le bronze.

Lancement, depuis Projet_Boreal/01_de : python ingestion/deposer_corpus.py
"""

import hashlib
import shutil
import sys
from pathlib import Path

PROJET = Path(__file__).resolve().parents[2]
SOURCE = PROJET / "sources" / "corpus"
BRONZE = PROJET / "plateforme" / "lac" / "bronze" / "corpus"

def empreinte(chemin: Path) -> str:
    return hashlib.sha256(chemin.read_bytes()).hexdigest()

def main():
    copies = identiques = 0
    for fichier in sorted(p for p in SOURCE.rglob("*") if p.is_file()):
        cible = BRONZE / fichier.relative_to(SOURCE)
        if cible.exists() and empreinte(cible) == empreinte(fichier):
            identiques += 1
            continue
        cible.parent.mkdir(parents=True, exist_ok=True)
        shutil.copyfile(fichier, cible)
        copies += 1
    print(f"Corpus : {copies} fichier(s) déposé(s), {identiques} déjà à jour, dans "
          f"{BRONZE.relative_to(PROJET).as_posix()}")

if __name__ == "__main__":
    sys.stdout.reconfigure(encoding="utf-8", errors="replace")
    main()
```

Lancez-le deux fois :

```
cd 01_de
python ingestion\deposer_corpus.py
python ingestion\deposer_corpus.py
cd ..
```

Corpus : 35 fichier(s) déposé(s), 0 déjà à jour, dans plateforme/lac/bronze/corpus
Corpus : 0 fichier(s) déposé(s), 35 déjà à jour, dans plateforme/lac/bronze/corpus

Le second passage ne recopie rien : il compare les contenus, pas les dates. Un dernier regard, dans DuckDB (duckdb) :

```
-- Attendu : 35 fichiers et 135 714 octets ; 33 documents au catalogue.  
ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot (READ_ONLY);  
USE entrepot;  
SELECT COUNT(*) AS fichiers, SUM(size) AS octets FROM read_blob('plateforme/lac/bronze/corpus/**');  
SELECT COUNT(*) AS documents FROM raw.corpus_index;
```

Les 35 fichiers sont les 33 documents, le catalogue index.csv et la note LISEZMOI.md de l'équipe de la documentation. ** descend dans les sous-dossiers. Quittez DuckDB (.exit).

Étape 5 — Versionner le chargeur et le dépôt du corpus

Le code va dans le dépôt ; les données et l'entrepôt, non. Depuis Projet_Boreal, dans l'Anaconda Prompt :

```
cd 01_de  
git add ingestion  
git status
```

git status liste les deux scripts de ingestion, charger_raw.py et deposer_corpus.py, prêts à être validés. L'entrepôt et le lac vivent dans plateforme/, hors du dépôt, et se reconstruisent en relançant les scripts.

```
git commit -m "Cours 03 : couche RAW et corpus déposé"  
git status  
cd ..
```

Le dernier git status répond que l'arbre de travail est propre.

Récapitulatif

Vous avez...	Preuve
vu pourquoi un chargement doit être idempotent	le même extrait chargé deux fois : 854 lignes pour 427 commandes
écrit et lancé le chargeur de la RAW	22 tables, 1 572 fichiers, 513 857 lignes ; « à jour » partout au second passage

Vous avez...	Preuve
inspecté la RAW	609 commandes aux lots 2023-03 et 2023-03_bis ; 158 146 messages pour 155 843 identifiants ; 5 623 objets pour 3 348 réclamations
déposé le corpus	35 fichiers, puis 0 au second passage
versionné	le chargeur et le dépôt du corpus dans 01_de

L'état du pipeline de Boréal à la fin du Cours 03 :

Couche	Ce qu'elle contient	Volume	Contrôle
bronze	l'historien (Cours 02)	92 fichiers, 275 436 lignes	CSV contre Parquet
bronze	la télémétrie (Cours 02)	843 fichiers, 158 146 messages	JSON contre Parquet
bronze	le corpus	35 fichiers	second dépôt : 0 fichier
entrepôt, raw	les 22 livraisons	513 857 lignes	second passage : 0 fichier
entrepôt, pipeline	le journal des chargements	1 572 fichiers	un fichier, une ligne

Prochaine étape (Cours 04). Les fichiers livrés sont dans la RAW. Au Cours 04, on va chercher ce qui ne se livre pas en fichiers : la base de l'ERP, lue par ODBC dans son conteneur, pour prouver que les extraits racontent la même histoire qu'elle, et la vraie API météo d'ECCC, page par page. Il commence par recevoir les fichiers de la plateforme et par monter la base de l'ERP.

Dépannage

Symptôme	Cause probable	Correctif
Catalog Error: SET schema: No catalog + schema named "entrepot" found.	DuckDB a été rouvert sans l'ATTACH	lancer ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot;; puis USE entrepot;
Catalog Error: Table with name "raw.erp_vbak" does not exist because schema "raw" does not exist.	même cause, ou le chargeur n'a pas encore tourné	l'ATTACH, puis l'étape 2 si besoin

Symptôme	Cause probable	Correctif
L'entrepôt est occupé : fermez la ligne de commande DuckDB, puis relancez., suivi de IO Error: Cannot open file	la ligne de commande DuckDB, ou un autre programme, tient boreal.duckdb ouvert	.exit dans DuckDB, puis relancer le script

Annexe — Tout le code, d'un seul tenant

Pour rejouer le labo sans relire les explications. Les commandes de terminal se tapent dans l'Anaconda Prompt, depuis Projet_Boreal ; les requêtes, dans la ligne de commande DuckDB.

1 — L'entrepôt et le second chargement (*Anaconda Prompt, puis DuckDB*)

```
mkdir plateforme\entrepot
duckdb

ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot;
USE entrepot;
CREATE SCHEMA IF NOT EXISTS essai;
CREATE OR REPLACE TABLE essai.vbak AS SELECT * FROM read_csv('sources/erp/extraits/2025-12/VBAK.csv', all_varchar = true,
filename = true);
INSERT INTO essai.vbak SELECT * FROM read_csv('sources/erp/extraits/2025-12/VBAK.csv', all_varchar = true, filename = true);
SELECT COUNT(*) AS lignes, COUNT(DISTINCT VBELN) AS commandes FROM essai.vbak;
DROP SCHEMA essai CASCADE;
.exit
```

2 — Le chargeur (*créer 01_de\ingestion\charger_raw.py avec le contenu de l'étape 2, puis*)

```
cd 01_de
python ingestion\charger_raw.py
python ingestion\charger_raw.py
cd ..
```

3 — Le corpus, et un regard sur la RAW (*créer 01_de\ingestion\deposer_corpus.py avec le contenu de l'étape 4, puis*)

```
cd 01_de
python ingestion\deposer_corpus.py
python ingestion\deposer_corpus.py
cd ..
duckdb
```

Puis, à l'invite D :

```
ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot (READ_ONLY);
USE entrepot;
SELECT COUNT(*) AS tables_raw FROM information_schema.tables WHERE table_schema = 'raw';
SELECT table_raw, COUNT(*) AS fichiers, SUM(lignes) AS lignes, COUNT(*) FILTER (WHERE lignes = 0) AS vides FROM
pipeline.chargements GROUP BY table_raw ORDER BY table_raw;
SELECT _lot, COUNT(*) AS en_tetes FROM raw.erp_vbak WHERE _lot LIKE '2023-03%' GROUP BY _lot ORDER BY _lot;
SELECT COUNT(*) AS messages, COUNT(DISTINCT message ->> 'message_id') AS identifiants FROM raw.telemetrie_sorties;
SELECT COUNT(*) AS fichiers, SUM(size) AS octets FROM read_blob('plateforme/lac/bronze/corpus/**');
.exit
```

4 — Le dépôt (Anaconda Prompt, depuis *Projet_Boreal*)

```
cd 01_de
git add ingestion
git commit -m "Cours 03 : couche RAW et corpus déposé"
git status
cd ..
```

Annexe — Sur Snowflake

Pour qui charge la RAW dans Snowflake plutôt que dans DuckDB. Les faits ont été vérifiés dans la documentation de Snowflake en septembre 2026 ; les commandes se tapent dans une feuille de calcul de Snowsight, sur votre propre compte. L'exemple charge VBAK ; chaque table suit le même patron.

Snowflake calcule dans le nuage : ses entrepôts ne voient pas les fichiers de votre disque. Un fichier doit donc d'abord entrer dans un stage, un espace de stockage que Snowflake connaît. Un stage interne est géré par Snowflake ; un stage externe pointe vers votre propre stockage objet (Amazon S3, Azure Blob Storage, Google Cloud Storage). En local, la question ne se pose pas : DuckDB lit les fichiers là où ils sont. Un stage n'a rien à voir avec la couche STAGING d'Analytics Engineering, malgré le mot.

```
USE ROLE SYSADMIN;
CREATE DATABASE IF NOT EXISTS BOREAL;
CREATE TRANSIENT SCHEMA IF NOT EXISTS BOREAL.RAW;
CREATE STAGE IF NOT EXISTS BOREAL.RAW.LIVRAISONS;
```

Pourquoi un schéma transient. Snowflake protège une table ordinaire deux fois : le voyage dans le temps (*Time Travel*), qui garde les versions passées pendant la durée choisie, puis sept jours de sécurité intégrée (*Fail-safe*), réservés à la reprise après sinistre par Snowflake lui-même, et facturés en stockage. Une table transient n'a pas de sécurité intégrée, et son voyage dans le temps dure 0 ou 1 jour. Toute table créée dans un schéma transient est transient. La RAW se reconstruit à partir des fichiers livrés, qui sont sa vraie copie de sûreté : payer la sécurité intégrée pour elle n'achète rien. Une couche qui est le seul exemplaire d'un travail, elle, garde la protection complète.

Pourquoi SYSADMIN, et pas un rôle plus fort. Dans Snowflake, le rôle qui crée un objet en devient propriétaire. Créer la base et le schéma avec SYSADMIN, le rôle prévu pour les objets de données, évite qu'ils appartiennent à ACCOUNTADMIN, que les autres rôles ne voient pas. En équipe, le chargement passe ensuite par un rôle dédié, qui n'a de droits que sur RAW : une erreur de script ne peut rien abîmer ailleurs.

Déposez les fichiers dans le stage, par Snowsight : menu Ingestion, Add Data, Load files into a Stage. Choisissez le schéma RAW et le stage LIVRAISONS, et précisez un chemin qui garde le mois, erp/extraits/2023-03/ par exemple : le lot se lira dans ce chemin. Un fichier y est limité à 250 Mo. Un client en ligne de commande (Snowflake CLI) fait le même dépôt par la commande PUT.

```
CREATE TABLE IF NOT EXISTS BOREAL.RAW.ERP_VBAK (
  MANDT VARCHAR, VBELN VARCHAR, ERDAT VARCHAR, ERZET VARCHAR, AEDAT VARCHAR, AUART VARCHAR,
  VKORG VARCHAR, VTWEG VARCHAR, KUNNR VARCHAR, VDATU VARCHAR, NETWR VARCHAR, WAERK VARCHAR,
  _FICHIER VARCHAR, _LOT VARCHAR, _CHARGE_LE TIMESTAMP_LTZ);

COPY INTO BOREAL.RAW.ERP_VBAK
FROM (
  SELECT $1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11, $12,
    METADATA$FILENAME, SPLIT_PART(METADATA$FILENAME, '/', -2), METADATA$START_SCAN_TIME
  FROM @BOREAL.RAW.LIVRAISONS)
PATTERN = '.*extraits/.*/VBAK[.]csv'
FILE_FORMAT = (TYPE = CSV SKIP_HEADER = 1 FIELD_OPTIONALLY_ENCLOSED_BY = ''');
```

Élément	Ce qu'il fait
\$1 ... \$12	les colonnes du CSV, par position
METADATA\$FILENAME	le chemin du fichier dans le stage : le _fichier du labo
SPLIT_PART(..., '/', -2)	l'avant-dernier morceau du chemin, le dossier du mois : le _lot ; un rang négatif compte depuis la fin
METADATA\$START_SCAN_TIME	l'heure du chargement : le _charge_le
PATTERN	une expression régulière sur le chemin : seulement les VBAK.csv des extraits
SKIP_HEADER = 1	saute l'en-tête : la valeur par défaut est 0

COPY INTO tient son propre journal. Pendant 64 jours, la table garde le nom, la taille et l'empreinte de chaque fichier chargé, et refuse de le recharger : relancer la commande ne double rien. L'option FORCE = TRUE passe outre et double les lignes ; LOAD_UNCERTAIN_FILES = TRUE charge les fichiers dont l'historique a expiré. Comme le journal du labo, celui de Snowflake laisse passer 2023-03_bis, qui porte un autre nom. Un champ vide devient NULL par défaut (EMPTY_FIELD_AS_NULL), comme dans la RAW locale.

Snowpipe charge les fichiers au fil de leur arrivée, sans qu'on relance COPY INTO. Il apprend l'arrivée d'un fichier par les notifications d'événements d'un stockage objet, ou par un appel de ses points d'accès REST, et garde son historique de chargement 14 jours. La formation Snowflake le traite en détail.

Annexe — Sur Databricks

Pour qui charge la RAW dans Databricks. Les faits ont été vérifiés dans la documentation de Databricks en septembre 2026. Databricks Free Edition suffit : calcul serverless seulement, un entrepôt SQL de taille 2X-Small, des quotas qui coupent le calcul pour le reste de la journée quand ils sont dépassés, et aucun usage commercial. Serverless veut dire que la plateforme fournit le calcul et le facture à l'usage, sans grappe de machines à dimensionner. Les commandes se tapent dans l'éditeur SQL, sur votre espace.

Les fichiers entrent dans un volume Unity Catalog, un espace de fichiers rangé dans un schéma, que les requêtes lisent par un chemin /Volumes/<catalogue>/<schéma>/<volume>/.

```
CREATE SCHEMA IF NOT EXISTS raw;
CREATE VOLUME IF NOT EXISTS raw.livraisons;
```

Déposez les dossiers erp/extraits/... dans le volume : menu New, Add or upload data, Upload files to a volume. Un fichier y est limité à 5 Go. Créez ensuite la table, vide et sans schéma, puis chargez :

```
CREATE TABLE IF NOT EXISTS raw.erp_vbak;

COPY INTO raw.erp_vbak
FROM (
  SELECT *, _metadata.file_path AS _fichier,
    element_at(split(_metadata.file_path, '/'), -2) AS _lot,
    current_timestamp() AS _charge_le
  FROM '/Volumes/<catalogue>/raw/livraisons/erp/extraits/')
FILEFORMAT = CSV
PATTERN = '*VBAK.csv'
FORMAT_OPTIONS ('header' = 'true', 'inferSchema' = 'false')
COPY_OPTIONS ('mergeSchema' = 'true');
```

Élément	Ce qu'il fait
_metadata.file_path	le chemin du fichier lu, par la colonne cachée _metadata : le _fichier du labo
element_at(split(...), -2)	l'avant-dernier morceau du chemin, le dossier du mois : le _lot

Élément	Ce qu'il fait
'inferSchema' = 'false'	aucune détection de types : toutes les colonnes arrivent en texte
'mergeSchema' = 'true'	la table vide prend la forme du premier chargement
PATTERN	un motif de noms de fichiers, relatif au chemin du FROM

COPY INTO est idempotent : il saute les fichiers déjà chargés, même s'ils ont été modifiés depuis. C'est la différence avec le chargeur du labo, qui recharge un fichier corrigé sous le même nom ; ici, il faut l'option 'force' = 'true', qui recharge tout et double les lignes. Pour un chargement continu, Auto Loader, une source de Structured Streaming nommée `cloudFiles`, suit les fichiers déjà traités dans son point de reprise et garantit que chacun n'est traité qu'une fois ; il lit aussi les volumes Unity Catalog. Pour les utilisateurs de SQL, Databricks recommande désormais les tables de diffusion en continu (*streaming tables*), qui reposent sur ce mécanisme.